

Build Plan: DMARC/BEC Email Security Tool

The stack, decided upfront

Piece	Choice	Why
Core language	Python	The only mature, battle-tested library for this exact job (<code>parsedmarc</code>) is Python. Also the language AI coding tools are strongest in.
Storage	SQLite	Zero setup, a single file, plenty for one domain or even a few hundred. Upgrade to Postgres later if needed.
Web dashboard	FastAPI + HTML templates + Chart.js	One language for backend and logic, plain HTML/CSS for the UI, Chart.js loaded from a CDN for graphs — no JavaScript build tools to fight with.
Report ingestion	IMAP via <code>parsedmarc</code> (Phase 1) → Cloudflare Email Worker (Phase 2, when scaling to other people's domains)	Start with the simplest path (a mailbox you already have); move to a fully automated pipeline once you have real users.
Hosting	Fly.io or Railway free tier, or a \$5/mo VPS	Enough for a personal tool and early users.
AI coding tool	Claude Code (terminal-based, can read/write files and run commands directly)	Best fit for a solo builder — it can scaffold, edit, test, and debug the whole project without you copy-pasting code back and forth.

Important boundary: no AI runs *inside* the finished product. AI tools help you *write* the code and the explanation text once, during development. The live product just runs plain Python logic — that's what makes it fast, predictable, and matches what you said you wanted.

Phase 1 — Set up your own domain as the test subject (no code)

What you're doing: Get your own domain sending DMARC reports to a mailbox you control, so you have real data to build against from day one.

Steps:

1. If you don't have one, buy a cheap domain and put its DNS on Cloudflare (free).

2. Add SPF and DKIM records for whatever you send email through (Gmail/Workspace, etc.).
3. Add a DMARC record at `p=none` with a `rua=mailto:` address pointing at a mailbox you can log into (a plain Gmail address is fine to start).

AI tool to use: Just chat with Claude (no coding tool needed yet) — tell it exactly what you send email through, and it will write the exact SPF/DKIM/DMARC record text for you to paste into Cloudflare's DNS panel.

Why this step first: Everything downstream needs real report data to test against. Reports start arriving within a day or two of publishing the record, so starting this immediately means you're not blocked later.

Phase 2 — Build the ingestion pipeline

What you're doing: Automatically pull the DMARC reports out of that mailbox and turn the raw XML into structured, queryable data.

Steps:

1. Open Claude Code in a new project folder and ask it to set up a Python project with `parsedmarc` installed (it's the standard open-source library for exactly this — actively maintained, handles IMAP, Gmail API, and Microsoft Graph mailbox connections, and parses both the old and new DMARC report formats).
2. Have Claude Code write a script that connects to your report mailbox over IMAP, pulls new reports, and parses them with `parsedmarc`.
3. Have it store each parsed record (sending IP, pass/fail on SPF and DKIM, whether it aligns with your domain, volume) into a SQLite database.
4. Set it to run on a schedule (a simple cron job or a loop with a daily sleep — no need for anything fancier yet).

AI tool to use: **Claude Code**, end to end. This is exactly the kind of "read the docs, wire up a library, write the glue code" task it's built for — give it the `parsedmarc` GitHub page and your IMAP credentials setup, and let it build and test the pipeline.

Why this matters: This is the unglamorous plumbing every competitor also has to build — but you're not writing an XML parser from scratch, you're standing on a proven library, so this phase should take a day or two, not weeks.

Phase 3 — Build the risk logic (the actual product)

What you're doing: Turn raw pass/fail data into an answer to the question that actually matters: *"Is it safe for me to turn on strict enforcement yet, and if not, what's stopping me?"*

Steps:

1. Group all the parsed records by sending source (IP address / sending service).
2. For each source, calculate: is this really you (passes SPF or DKIM and aligns with your domain), or is it unauthenticated/spoofed traffic?
3. Compute a plain percentage: what fraction of your total mail volume would survive if you turned on `p=reject` today.
4. Write a simple threshold-based readiness score (e.g., "98%+ aligned for 2+ weeks straight = safe to move to quarantine").

AI tool to use: Claude Code to write this as plain Python logic (a `pandas` dataframe or even just dictionaries — no need for anything heavier). Also useful: paste in a few real parsed reports and ask Claude to help you sanity-check edge cases (a forwarded email, a third-party sender like Mailchimp, etc.) so your scoring logic doesn't break on real-world messiness.

Why this is the real product: Anyone can display raw pass/fail numbers — dmarcian and EasyDMARC already do that. The actual value is turning it into a clear, safe, "do this next" recommendation, which is just good rule design, not AI.

Phase 4 — Write the plain-English explanations (once, by hand)

What you're doing: For every type of finding your logic can produce ("this IP is sending unauthenticated mail claiming to be you," "this is a known service you forgot to authorize," etc.), write one clear, non-jargon explanation and a concrete next step.

Steps:

1. List every distinct finding type your Phase 3 logic can output.
2. For each one, draft: what it means in plain language, why it matters, and exactly what to click/change to fix it.
3. Store these as a lookup table/templates in your code — the app fills in the specific domain, IP, or date, but the wording itself is fixed content you wrote.

AI tool to use: Claude (chat) — this is a writing task, not a coding task. Describe each technical finding and ask Claude to help you draft the clearest possible plain-English version. You review and finalize the wording once; it never changes at runtime.

Why do it this way instead of calling an AI live: This keeps the product's answers 100% predictable and free — no API costs per user, no risk of a bad or inconsistent explanation, and it matches your "no AI in the product" requirement exactly.

Phase 5 — Build the dashboard

What you're doing: A simple web page where you (and later, other users) can see their score, their flagged senders, and their plain-English action list.

Steps:

1. Ask Claude Code to scaffold a FastAPI app with a couple of routes: a summary page and a detail page per flagged sender.
2. Use plain HTML templates (Jinja2, which ships with FastAPI's ecosystem) — no separate frontend framework needed yet.
3. Add one chart (pass/fail trend over time) using Chart.js pulled from a CDN link — a few lines of JavaScript, no build step.

AI tool to use: Claude Code for all of it — backend routes, templates, and the small chart script. This phase is mostly "wire together things that already exist," which is where Claude Code is fastest.

Why keep it this simple: A dashboard that clearly shows one number and one action list beats a feature-heavy one nobody understands. You can always add more views later once you know what people actually want to see.

Phase 6 — Deploy it and point it at your real domain

Steps:

1. Ask Claude Code to write a `Dockerfile` and a config file for Fly.io or Railway (both have usable free tiers for a small personal app).
2. Deploy, set your environment variables (mailbox credentials, etc.) as secrets.
3. Point it at the real inbox from Phase 1 and let it run.

AI tool to use: Claude Code for the deployment config; ask it to explain any error messages you hit along the way rather than debugging blind.

Phase 7 — Run it on yourself for a few weeks

No new building here — just use it. Let real reports accumulate, watch what the dashboard tells you, and note anything confusing or missing. This is where your next real features come from, not from guessing.

Phase 8 — Scaling later (once you're ready for other people's domains)

What changes:

- **Ingestion:** move from "you personally check an IMAP inbox" to a **Cloudflare Email Worker** (there's an open-source reference implementation, `cloudflare/dmarc-email-worker`, that already does this) so any customer can point their `rua=` address at your service directly — free, and it scales to many domains without you managing mailboxes.
- **New language needed:** the Cloudflare Worker is written in JavaScript/TypeScript —

this is the one place you'll step outside Python.

- **Billing:** add Stripe for subscriptions once you have paying users.
- **Login:** add simple auth (magic-link email or Google OAuth login) so each customer only sees their own domains.

AI tool to use: Claude Code again for the Worker script and Stripe integration — same workflow, just a new language it's equally capable in.

Why wait to add this: none of it is needed to prove the idea works on yourself and a few friends. Building it earlier just adds complexity before you know it's worth it.

Quick summary table

Phase	What you build	Language	AI tool
1	DNS records	—	Claude (chat)
2	Report ingestion	Python	Claude Code
3	Risk scoring logic	Python	Claude Code
4	Plain-English content	— (writing)	Claude (chat)
5	Dashboard	Python + HTML/JS	Claude Code
6	Deployment	Docker/config	Claude Code
7	Real-world use	—	—
8	Multi-tenant scaling	+ JavaScript/TypeScript	Claude Code